

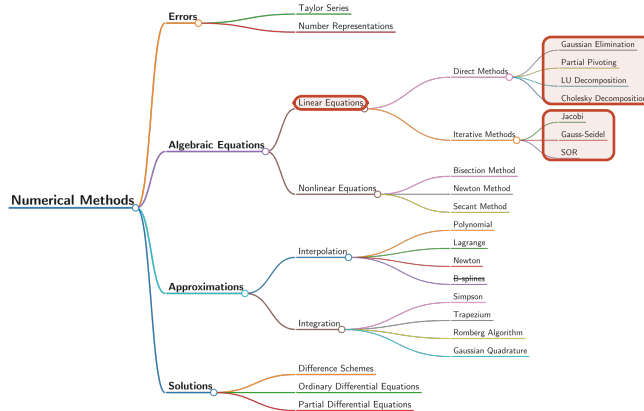
Linear Equations

CTMS-MAT-13 • Direct and iterative solution of $A\vec{x} = \vec{b}$

Dr. David Sinden

September 10, 2026

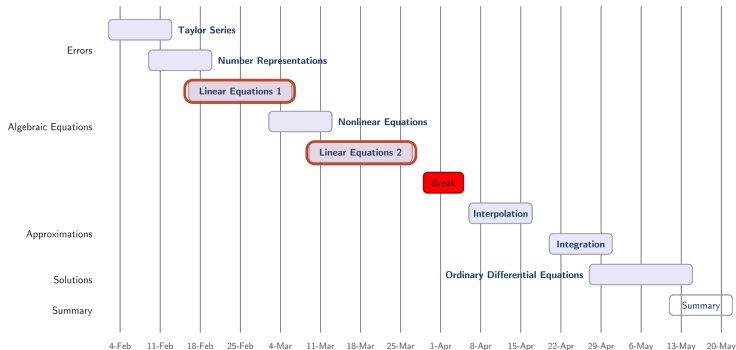
Where this sits in the course



Full outline: djps.github.io/docs/numericalmethods/intro/intro/

Schedule

Schedule 2026



Notes: djps.github.io/teaching/numerical_methods.pdf

Today

Linear systems

The problem

$$A\vec{x} = \vec{b}, \quad b_i = \sum_{j=1}^n a_{ij}x_j$$

- $A \in \mathbb{R}^{m \times n}$ — coefficients;
- $\vec{x} \in \mathbb{R}^n$ — unknowns;
- $\vec{b} \in \mathbb{R}^m$ — data.

$m = n$ makes A **square**; we assume this and a unique solution throughout.

Linearity means

$$A(\alpha\vec{x} + \beta\vec{y}) = \alpha A\vec{x} + \beta A\vec{y}$$

for all $\alpha, \beta \in \mathbb{R}$ — every unknown appears to the first power only.

Data vectors are rarely two-dimensional: a patient record $\vec{v} = (\text{weight, height, temp, pulse, HRV}) \in \mathbb{R}^5$, and $n > 10^4$ is routine.

Two families

Direct — finite sequence of operations, exact in exact arithmetic.

- Gaussian elimination
- LU decomposition
- Cholesky decomposition

Cost $\mathcal{O}(n^3)$.

Iterative — a sequence $\vec{x}_1, \vec{x}_2, \dots$ converging to the solution.

- Jacobi, Gauss–Seidel
- SOR, conjugate gradient

Cost $\mathcal{O}(n^2)$ per iteration.

TAKEAWAY

For $n > 10^4$ an $\mathcal{O}(n^3)$ method is unaffordable — and for sparse A the factors of a direct method are not even sparse.

Gaussian elimination

Elimination and back substitution

Transform the augmented matrix $[A \mid \vec{b}]$ to **upper triangular** form, then solve from the bottom up.

Three **elementary row operations** do not change the solution:

1. swap: $r_i \leftrightarrow r_j$;
2. scale: $r_i \leftarrow \alpha r_i, \alpha \neq 0$;
3. combine: $r_i \leftarrow r_i - \ell_{ik} r_k$.

Elimination is $\mathcal{O}(n^3)$; back substitution only $\mathcal{O}(n^2)$.

The algorithm

For $k = 1, \dots, n - 1$ (columns being eliminated):

$$\ell_{ik} = \frac{a_{ik}}{a_{kk}}, \quad i = k + 1, \dots, n$$

$$a_{ij} \leftarrow a_{ij} - \ell_{ik} a_{kj}, \quad i, j > k$$

$$b_i \leftarrow b_i - \ell_{ik} b_k, \quad i > k$$

then back-substitute:

$$x_n = \frac{b_n}{a_{nn}}, \quad x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=i+1}^n a_{ij} x_j \right).$$

The divisor a_{kk} is the **pivot**. Everything that can go wrong, goes wrong here.

Why the pivot matters

$$\varepsilon x_1 + x_2 = 1$$

$$x_1 + x_2 = 2$$

with $0 < \varepsilon \ll 1$. Using ε as pivot multiplies row 1 by $1/\varepsilon$ before subtracting: the computed x_2 carries an error amplified by $1/\varepsilon$.

The fix

Swap the rows first. The pivot becomes 1, the multiplier ε , and nothing is amplified.

Note the system itself is perfectly well conditioned — only the *algorithm* was unstable.

TAKEAWAY

A small pivot amplifies round-off. Pivoting is not an optimisation; it is what makes elimination usable.

Scaled partial pivoting

1. Compute the row scales once: $s_i = \max_{j=1,\dots,n} |a_{ij}|$.
2. For each pivot column $r = 1, \dots, n - 1$, choose the row $i \geq r$ maximising

$$\left| \frac{a_{ir}}{s_i} \right|.$$

3. Swap rows r and i , then eliminate.

Dividing by s_i compares pivots *relative* to the size of their own row, so a row measured in different units cannot hijack the choice.

Factorisations

LU decomposition

Elimination is a factorisation in disguise: the multipliers ℓ_{ik} are exactly the entries of a lower triangular L , and the reduced matrix is U .

$$A = LU, \quad L \text{ unit lower triangular, } U \text{ upper triangular.}$$

Solving, once L and U are known

$$A\vec{x} = \vec{b} \implies L\vec{y} = \vec{b} \text{ (forward substitution), } U\vec{x} = \vec{y} \text{ (back substitution).}$$

The $\mathcal{O}(n^3)$ cost is paid once; each further right-hand side costs $\mathcal{O}(n^2)$.

Cholesky decomposition

If A is

1. **symmetric**, $a_{ij} = a_{ji}$, and
2. **positive definite**, all eigenvalues > 0 ,

then

$$A = LL^T$$

with L lower triangular and $\ell_{ii} > 0$.

$$\ell_{ij} = \frac{1}{\ell_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} \ell_{ik} \ell_{jk} \right), \quad i > j$$
$$\ell_{ii} = \sqrt{a_{ii} - \sum_{k=1}^{i-1} \ell_{ik}^2}$$

A negative radicand proves A is *not* positive definite — the algorithm is also a test.

TAKEAWAY

Half the storage, half the work of LU, no pivoting needed for stability — symmetry and definiteness are worth exploiting whenever you have them.

Cholesky: a small example

$$A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 6 \end{pmatrix} \quad \Rightarrow \quad L = \begin{pmatrix} 1 & & \\ 1 & 1 & \\ 1 & 2 & 1 \end{pmatrix}$$

Symmetric, positive diagonal, and every radicand positive — so $A = LL^T$ and A is positive definite.

Check the middle entry: $\ell_{22} = \sqrt{2 - 1^2} = 1$; and $\ell_{32} = (3 - 1 \cdot 1)/1 = 2$,
 $\ell_{33} = \sqrt{6 - 1 - 4} = 1$.

Iterative methods

Approximating $A^{-1}\vec{b}$, not A^{-1}

Rather than inverting A , generate a sequence $\vec{x}_1, \vec{x}_2, \dots \in \mathbb{R}^n$ approaching the solution. For non-singular A and Q , take

$$Q\vec{x}_{k+1} = (Q - A)\vec{x}_k + \vec{b}, \quad k = 0, 1, 2, \dots$$

from a given $\vec{x}_0$, or equivalently

$$\vec{x}_{k+1} = (I - Q^{-1}A)\vec{x}_k + Q^{-1}\vec{b}.$$

Rearranged, $Q(\vec{x}_{k+1} - \vec{x}_k) = \vec{b} - A\vec{x}_k$: the update is driven by the **residual**, and vanishes exactly at the solution. Any non-singular Q therefore gives a **consistent** scheme — but not necessarily a convergent one.

What to ask of Q

- (i) Q^{-1} must be cheap — otherwise nothing has been gained.
- (ii) Q should approximate A well, so that $I - Q^{-1}A$ is small.
- (iii) These pull in opposite directions: $Q = A$ is the best approximation and the worst to invert.

The other extreme, $Q = I$, gives **Richardson iteration**

$$\vec{x}_{k+1} = (I - A)\vec{x}_k + \vec{b},$$

free to apply, and convergent only sometimes.

TAKEAWAY

Every iterative method in this lecture is one choice of Q on the line between I and A .

Convergence

Theorem

The iteration $\vec{x}_{k+1} = (I - Q^{-1}A)\vec{x}_k + Q^{-1}\vec{b}$ converges for every initial guess $\vec{x}_0$ if and only if

$$\rho(I - Q^{-1}A) < 1, \quad \rho(M) = \max_{i=1,\dots,n} |\lambda_i(M)|,$$

the **spectral radius** of M .

So convergence is determined by Q — consistency is not. And ρ sets the rate: the error falls by a factor ρ per iteration.

Proof

Let $\vec{x}$ solve $A\vec{x} = \vec{b}$ and $\vec{e}_k = \vec{x}_k - \vec{x}$. Since $\vec{x} = (I - Q^{-1}A)\vec{x} + Q^{-1}\vec{b}$, subtracting the iteration gives

$$\vec{e}_k = (I - Q^{-1}A)\vec{e}_{k-1} \implies \vec{e}_k = (I - Q^{-1}A)^k \vec{e}_0.$$

For $\vec{e}_k \rightarrow \vec{0}$ from any $\vec{e}_0 \neq \vec{0}$ we need $M^k \rightarrow 0$, where $M = I - Q^{-1}A$; and

$$M^k \rightarrow 0 \in \mathbb{R}^{n \times n} \iff \rho(M) < 1,$$

seen by diagonalising M (each eigenvalue contributes λ_i^k). □

In norms, $\|\vec{e}_k\| \leq \|M\|^k \|\vec{e}_0\|$ by submultiplicativity — the induced matrix norm bounds the decay.

Jacobi, Gauss–Seidel, SOR

Three choices of Q

Split $A = L + D + U$: strictly lower, diagonal, strictly upper.

Jacobi

Gauss–Seidel

SOR

$$Q = D$$

$$Q = L + D$$

$$Q = L + \frac{1}{\omega}D$$

Trivial inverse, $1/a_{ii}$ on the diagonal. Fully parallel.

One forward substitution per step; uses updated values immediately.

A **relaxation parameter** $\omega > 0$ tunes between them; $\omega = 1$ is Gauss–Seidel.

TAKEAWAY

More of A inside Q means fewer iterations and more work per iteration. The right balance is problem-dependent.

Convergence conditions

Jacobi

If A is **strictly** diagonally dominant, $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for every row i , then Jacobi converges for any $\vec{x}_0$. (Sufficient, not necessary.)

Gauss–Seidel

Diagonal dominance (not necessarily strict) suffices.

SOR

If A is symmetric with positive diagonal and $\omega \in (0, 2)$, SOR converges if and only if A is positive definite.

Example

$$A = \begin{pmatrix} 2 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} 1 \\ 8 \\ 5 \end{pmatrix}, \quad \vec{x} = \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix}.$$

Jacobi, $Q = D = \text{diag}(2, 3, 2)$:

$$I - D^{-1}A = \begin{pmatrix} 0 & 1/2 & 0 \\ 1/3 & 0 & 1/3 \\ 0 & 1/2 & 0 \end{pmatrix},$$

$$D^{-1}\vec{b} = \left(\frac{1}{2}, \frac{8}{3}, \frac{5}{2}\right)^{\top}.$$

From $\vec{x}_0 = \vec{0}$: $\vec{x}_1 = \left(\frac{1}{2}, \frac{8}{3}, \frac{5}{2}\right)^{\top}$, and
 $\vec{x}_{k+1} = (I - D^{-1}A)\vec{x}_k + D^{-1}\vec{b}$.

Note $\rho = 1/\sqrt{3} \approx 0.577$: roughly one extra correct digit every four iterations.

Example: the three methods compared

Gauss-Seidel, $Q = L + D$:

$$Q = \begin{pmatrix} 2 & & \\ -1 & 3 & \\ 0 & -1 & 2 \end{pmatrix}, \quad Q^{-1} = \begin{pmatrix} 1/2 & 0 & 0 \\ 1/6 & 1/3 & 0 \\ 1/12 & 1/6 & 1/2 \end{pmatrix}$$

$$I - Q^{-1}A = \begin{pmatrix} 0 & 1/2 & 0 \\ 0 & 1/6 & 1/3 \\ 0 & 1/12 & 1/6 \end{pmatrix}$$

The first column is always zero: x_1 is final after one sweep.

method	iterations
Jacobi	21
Gauss-Seidel	9
SOR, $\omega = 1.1$	7

to reach $(2, 3, 1)^\top$ from $\vec{x}_0 = \vec{0}$.

For SOR,

$$Q = \begin{pmatrix} 20/11 & & \\ -1 & 30/11 & \\ 0 & -1 & 20/11 \end{pmatrix} \text{ and}$$

the first column of $I - Q^{-1}A$ is *not* zero.

Consistency, stability, convergence

Theorem (Lax–Richtmyer)

A numerical scheme is convergent if it is consistent and stable.

Consistent — $\vec{x}_k \rightarrow \vec{r}$ implies $A\vec{r} = \vec{b}$:
the scheme solves the right problem.

Stable — errors do not grow:
 $\|\vec{e}_k\| \leq \|\vec{e}_0\|$.

TAKEAWAY

Consistency is about the equation; stability is about the algorithm. Neither alone gives an answer.

Direct methods spend n^3 once.
Iterative methods spend n^2
as often as you let them.

Summary

- Gaussian elimination reduces $[A \mid \vec{b}]$ to triangular form in $\mathcal{O}(n^3)$, then back-substitutes in $\mathcal{O}(n^2)$.
- Pivot choice, not the elimination itself, decides stability; scaled partial pivoting compares candidates relative to their own row.
- LU stores the elimination so extra right-hand sides cost $\mathcal{O}(n^2)$; Cholesky halves that again for symmetric positive definite A .
- Iterative methods choose a cheap $Q \approx A$ and iterate $\vec{x}_{k+1} = (I - Q^{-1}A)\vec{x}_k + Q^{-1}\vec{b}$; convergence for all $\vec{x}_0$ holds iff $\rho(I - Q^{-1}A) < 1$.
- $Q = D$, $L + D$, $L + D/\omega$ give Jacobi, Gauss–Seidel and SOR — increasing cost per step, decreasing step count.

Take-home messages

1. **Never invert a matrix.** Factor it, or iterate. $A^{-1}\vec{b}$ is a solve, not a product.
2. **Always pivot.** A well-conditioned system solved without pivoting can still return nonsense.
3. **Exploit structure.** Symmetry, definiteness, sparsity and bandedness each buy an order of magnitude — and direct factorisation destroys sparsity.
4. **Size chooses the family.** Small and dense: factorise. Large and sparse: iterate, and judge Q by the spectral radius it produces.

Next: nonlinear equations.